

[H-01] The function `Registry::register` doesn't resend the excess ETH

Description:

The sole purpose of the register function is to register with a specific amount of ether 1 ether, and the a user that registers with excess will be sent back excess amount of ether sent.

However this function doesn't do that, it doesn't encode for it user to receive excess ether.

Impact:

In a scenario that a user will send ether greater than the price to register 1 ether the contract still holds all that ether.

Proof of Concept:

```
function test_fuzzRegister(uint256 amountToPay) public {
    uint256 minAmount = registry.PRICE();
    amountToPay = bound(amountToPay, minAmount, MAX_PRICE);

    vm.deal(alice, amountToPay);
    vm.startPrank(alice);

    uint256 aliceBalanceBefore = address(alice).balance;

    registry.register{value: amountToPay}();

    uint256 aliceBalanceAfter = address(alice).balance;
    uint256 balanceOfRegisterAfter = address(registry).balance;
    vm.stopPrank();

    console.log("aliceBalanceBefore", aliceBalanceBefore);
    console.log("aliceBalanceAfter", aliceBalanceAfter);
    console.log("balanceOfRegisterAfter", balanceOfRegisterAfter);
    assertEq(aliceBalanceAfter, aliceBalanceBefore - registry.PRICE(), "Unexpected user bal");
}
```

This test reverts

```

[0] console::log("aliceBalanceBefore", 17446744073709554914 [1.744e19]) [staticcall]
|   └─ ← [Stop]
└─ [0] console::log("aliceBalanceAfter", 0) [staticcall]
|   └─ ← [Stop]
└─ [0] console::log("balanceOfRegisterAfter", 17446744073709554914 [1.744e19]) [staticcall]
|   └─ ← [Stop]
└─ [325] Registry::PRICE() [staticcall]
|   └─ ← [Return] 1000000000000000000 [1e18]
└─ [0] VM::assertEq(0, 16446744073709554914 [1.644e19], "Unexpected user balance") [staticcall]
|   └─ ← [Revert] Unexpected user balance: 0 != 16446744073709554914
└─ ← [Revert] Unexpected user balance: 0 != 16446744073709554914

```

Recommended Mitigation:

The recommended mitigation will be to add this line of code

```

function register() external payable {
    if(msg.value < PRICE) {
        revert PaymentNotEnough(PRICE, msg.value);
    }
+   if(msg.value > PRICE ) {
+       uint256 differenceToSendBack = msg.value - PRICE;
+       (bool success, ) = payable(msg.sender).call{value: differenceToSendBack}("");
+       if(!success) {
+           revert Registry__SendFailed();
+       }
+   }

    registry[msg.sender] = true;
}

```